Fauzan Arrizqy Putra

Logic Test Bagian A

1. **Race Condition di Sisi Frontend (React)**
    a. Pengguna akan melihat hasil output yang salah, karena ketiga request dikirim hampir bersamaan saat pengguna mengetik dengan cepat. Karena jaringan tidak stabil, response untuk request “a” keluar terakhir. Pada kode bagian setResults(data.data) dijalankan secara urut sesuai tiba response. Karena respons untuk “a” tiba paling akhir, results nya akan ditimpa dengan hasil dari query “a”, mengabaikan hasil “ap” dan “app” yang sebenarnya diketik lebih dulu.
    b. Dalam konteks data fetching di frontend istilah umum ini dikenal dengan race condition.
    c. Berikut perbaikannya menggunakan AbortController :

```
function useSearchResults(query: string) {
  const [results, setResults] = useState<Announcement[]>([]);
  useEffect(() => {
    if (!query) {
      setResults([]);
      return;
    }
const controller = new AbortController();
    fetch(`/api/announcements?search=${query}`,{ signal: controller.signal })
      .then((res) => res.json())
      .then((data) => setResults(data.data));
.catch(error) => {
        if (error.name !== 'AboutError'){
                console.error('Fetch error: ', error);
}});
return () => {
        controller.abort();
};
  }, [query]);
  return results;
}
```

    Dengan menggunakan Abort controller setiap kali pengguna mengetik huruf baru, useEffect akan menjalankan fungsi cleanup untuk membatalkan network request yang sedang berjalan dari query sebelumnya.

2. Race Condition di Sisi Backend (Go)
    a. Walaupun logika kode terlihat benar, kode ini mengalami race condition karena ada jeda waktu antara proses pengecekkan dan proses eksekusi penyimpanan.
    b. Berikut penjelasan dua cara berbeda:
        i. Level Database: Pada level ini bisa menggunakan database constraint & pessimistic locking dengan cara mengunci baris data course tertentu menggunakan select di dalam sebuah transaksi database, atau atomic update dengan cara mengubah query increment di database agar memiliki kondisi langsung.
        ii. Level Aplikasi: Menggunakan distributed lock seperti redis/redlock. Menerapkan mekanisme ini untuk menggunakan readis sebelum masuk ke logika usecase pendaftaran berdasarkan courseID.
    c. Untuk sistem seperti SIAKUP saya memilih untuk menggunakan level database menggunakan atomic conditional update, karena dengan menggunakan cara ini dapat menjamin konsistensi data meskipun aplikasi berjalan di banyak pod tanpa perlu mengelola status lock yang kompleks.

3. Struktur Data Hirarkis (Typescript)

```
interface OrgUnit {
  id: string;
  parentId: string | null; // null jika unit ini adalah root (fakultas)
  name: string;
}

interface OrgNode {
  id: string;
  name: string;
  children: OrgNode[];
}

function buildTree(units: OrgUnit[]);
OrgNode[]{
  const map = new Map<string,OrgNode>();
  units.forEach(unit =>{
    map.set(unit.id, {
        id: unit.id,
        name: unit.name,
        children: []
});
});
  const roots: OrgNode[] = [];

units.forEach(unit =>{
  const node - map.get(unit.id)!;

  if(unit.parentId !== null && map.has(unit.parentId)){
        const parentNode = map.get(unit.parentId)!;
                parentNode.children.push(node);

}else{
        roots.push(node);
}
});
return roots;
}
```